MuseAI LangGraph Submission Report

LangGraph integration into an existing CrewAI-based project with shared frontend support, defined workflow steps, and screenshot-ready documentation.

Student	Mert Kedik
Project	MuseAI - Creative Engine
GitHub Account	https://github.com/Mertlimert
Repository	https://github.com/Mertlimert/MuseAI-Creative-Engine.git
Git Account / Repo Link	https://github.com/Mertlimert/MuseAI-Creative-Engine.git
Relevant Commit	23e2172 - Add LangGraph encounter workflow alongside CrewAI
Report Date	2026-04-20

1. Assignment Coverage

This report documents how LangGraph was added into the existing MuseAI project without creating a separate application. The original CrewAI implementation was preserved, and both orchestration approaches now work in the same encounter system.

- LangGraph was integrated into the current project instead of starting a new one.
- CrewAI support was kept active, and the user can choose between CrewAI and LangGraph from the same interface.
- The LangGraph workflow was implemented with clearly defined steps and a shared state model.
- Git history includes a dedicated integration commit and the repository link is included in this report.
- LangSmith support was prepared through environment variables for traceability and debugging.
- Screenshots in this report document the implemented features and the live encounter flow.

2. Project Purpose and Why LangGraph Was Added

MuseAI is a storytelling and RPG content application that already supported an encounter mode powered by CrewAI. LangGraph was introduced to demonstrate a second orchestration style focused on stateful workflows, explicit node transitions, and conditional execution. This made the project a stronger example for classroom discussion because the same frontend experience can now be backed by either agent-based orchestration or graph-based control.

- CrewAI is useful for role-based multi-agent execution.
- LangGraph is useful when execution steps must be explicit, inspectable, and state-driven.
- The project now allows direct comparison between two orchestration strategies while preserving one shared UI and one shared API contract.

3. Implementation Summary

The integration was completed by extending both the backend and the frontend. On the backend, a new LangGraph encounter engine was added, shared request and response models were aligned, and the

API was updated to dispatch to the selected engine. On the frontend, an engine selector was introduced so the user can run the same D&D; encounter with CrewAI or LangGraph while keeping the rest of the user experience unchanged.

- Added **backend/langgraph_engine.py** to define the graph nodes and execution flow.
- Updated **backend/api.py** so requests can route to either CrewAI or LangGraph.
- Aligned **backend/schemas.py** so both engines return the same frontend-friendly response shape.
- Updated **app.js**, **index.html**, and **style.css** so the user can select the active orchestration engine from the encounter interface.
- Prepared LangSmith-ready configuration through environment variables in the backend setup.

4. LangGraph Workflow Steps

The LangGraph implementation was intentionally designed with well-defined steps so that its execution is understandable during demonstration and easy to explain in class. Each step has a clear responsibility and passes structured state to the next node.

- **Step 1 - Prepare Context:** The graph gathers player stats, story premise, campaign history, NPC histories, the latest player action, and location context into a shared encounter state.
- **Step 2 - Game Master Node:** The Game Master node evaluates the action and produces a structured output containing the narration and a list of NPCs that should react.
- **Step 3 - Conditional NPC Branch:** If at least one NPC must respond, the graph routes execution into the NPC dialogue node; otherwise, it skips directly to finalization.
- **Step 4 - Final Response Assembly:** The graph combines narration, NPC reactions, engine metadata, and workflow steps into a payload that the frontend can render immediately.

5. CrewAI and LangGraph Together

A key requirement of the assignment was to add LangGraph alongside the current system instead of replacing it. This requirement is satisfied. CrewAI remains available for the original multi-agent approach, while LangGraph is available for stateful step orchestration. Both engines are accessible from the same live encounter screen, both operate on the same character and location context, and both return compatible data to the frontend.

- No new project was created.
- The same application supports both CrewAI and LangGraph.
- The same player action flow works with either engine.
- The same frontend chat area can render the output of both systems.

6. LangSmith Integration

LangSmith integration was prepared as a helpful tracing option for development and debugging. With LangSmith enabled, it becomes possible to inspect which nodes executed, how state moved through the graph, and whether an NPC branch was triggered during a specific encounter run. This is especially useful when discussing implementation details in class because it makes the graph behavior visible beyond the frontend.

- LANGCHAIN_TRACING_V2=true
- LANGCHAIN_API_KEY=*your_key_here*
- LANGCHAIN_PROJECT=MuseAI-LangGraph

7. Files Added or Updated

Area	Files
Backend	backend/langgraph_engine.py, backend/api.py, backend/crew_engine.py, backend/schemas.py, bac
Frontend	index.html, app.js, style.css
Documentation	LangGraph_Report.md and this final submission PDF

8. Screenshot Evidence

The following screenshots were selected from the updated LangGraph demo session stored under assets/report. Older CrewAI report images were intentionally excluded so that this PDF reflects the latest LangGraph-specific walkthrough.

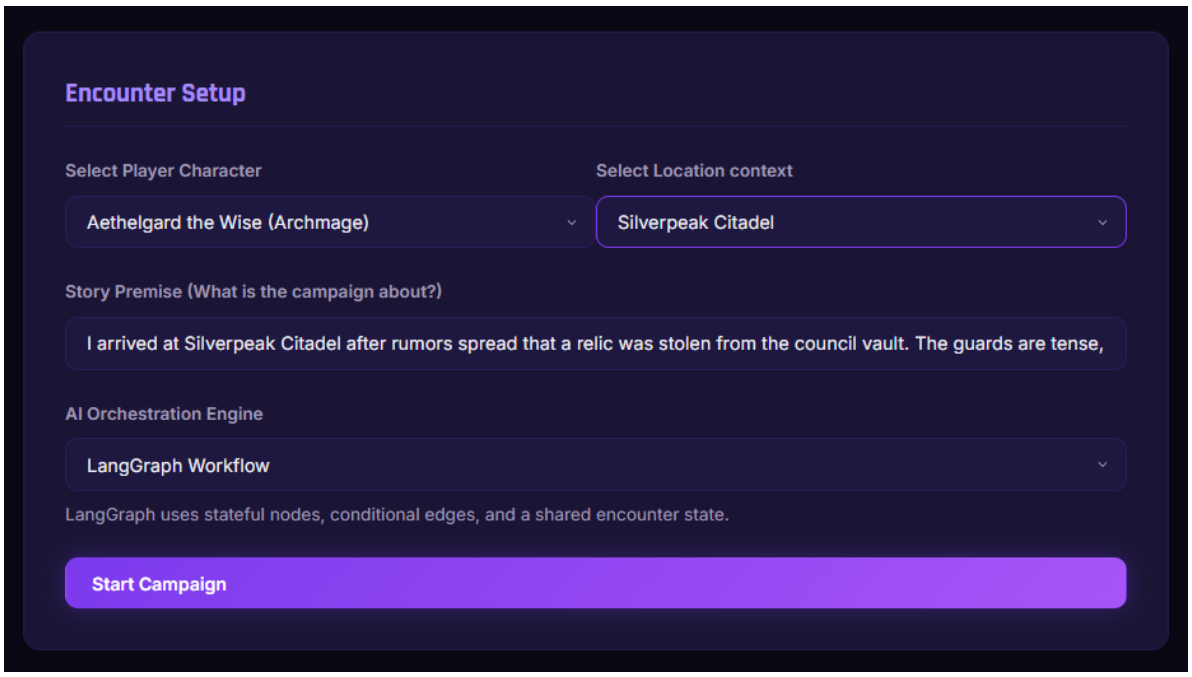

Figure 1. Encounter setup screen with Aethelgard the Wise, Silverpeak Citadel, and the LangGraph Workflow engine selected. This proves that LangGraph was integrated into the existing project interface instead of being developed as a separate application.

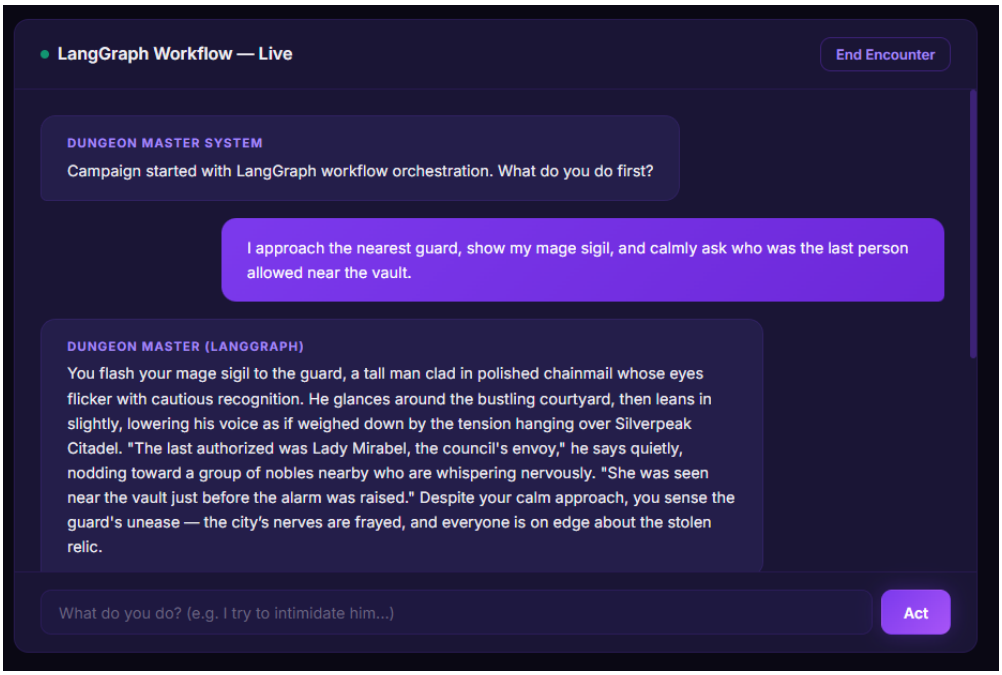

Figure 2. First LangGraph response after the player's opening question to the guard. The Game Master narration is rendered directly in the shared chat interface.

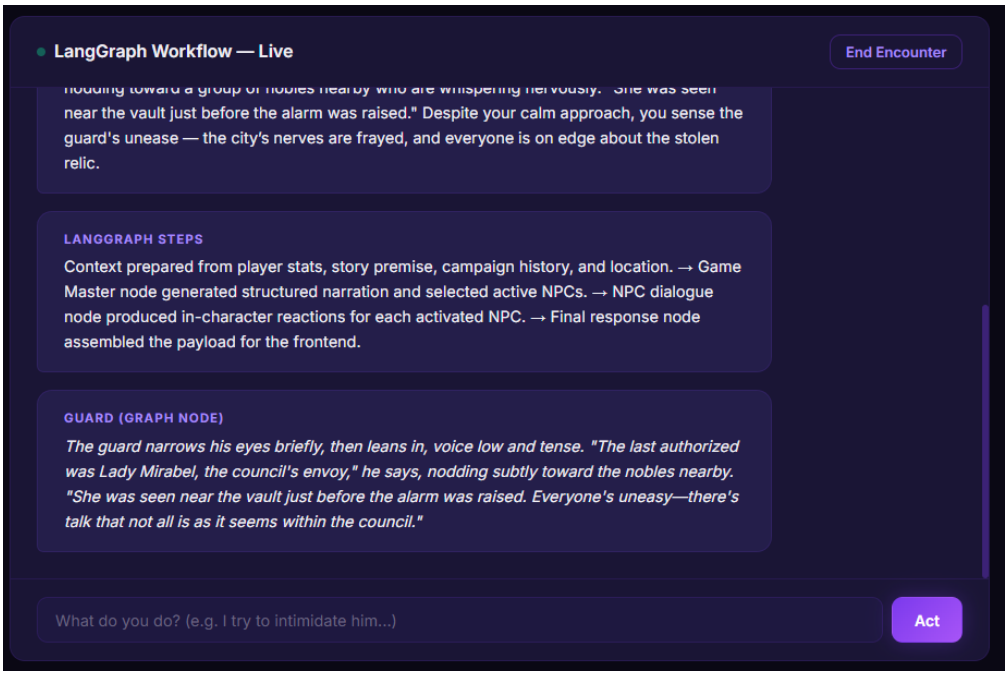

Figure 3. The same first interaction showing the LangGraph workflow steps and the Guard node response. This demonstrates explicit step visibility and conditional NPC-style output in one screen.

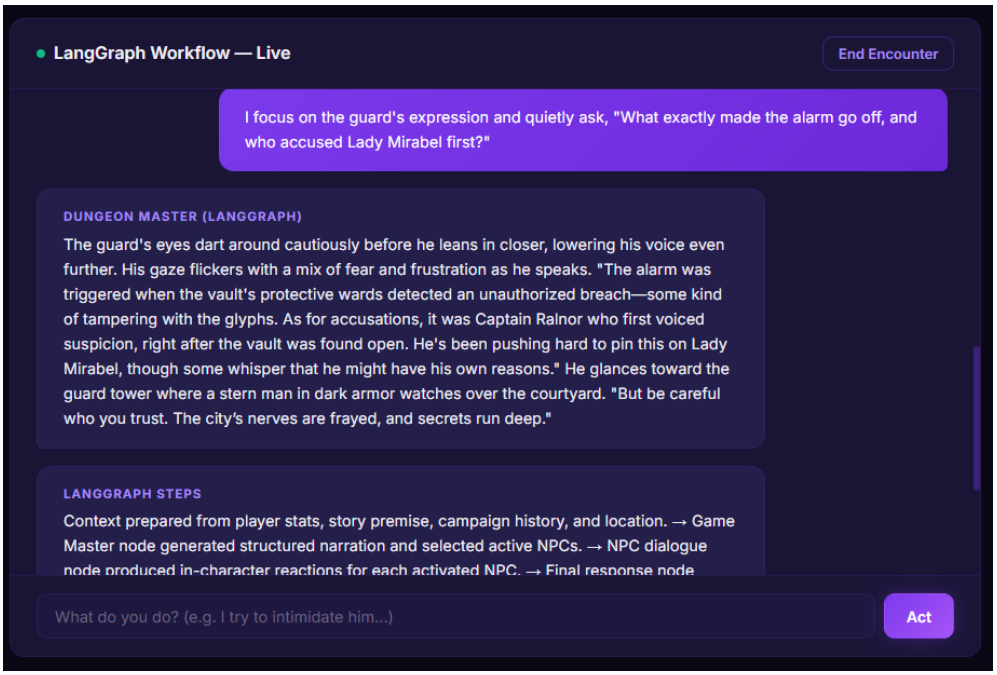

Figure 4. Second interaction in which the player asks about the alarm and the first accusation. The LangGraph-driven Game Master continues the investigation while preserving encounter context.

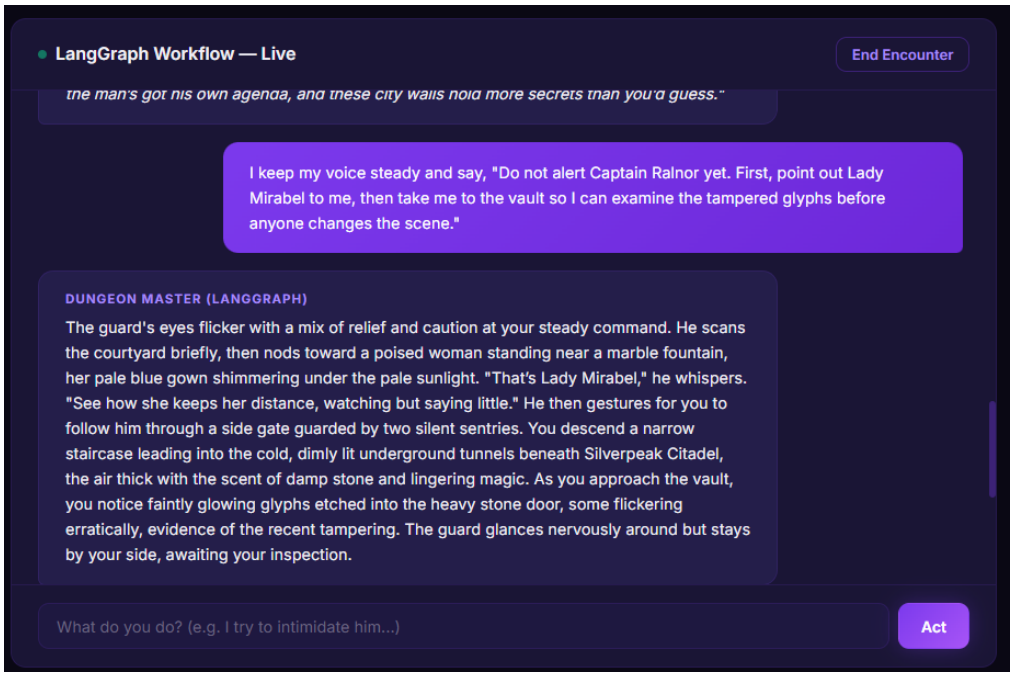

Figure 5. Continuation of the second interaction showing workflow steps and the Guard node's in-character explanation, supporting the graph's shared-state execution model.

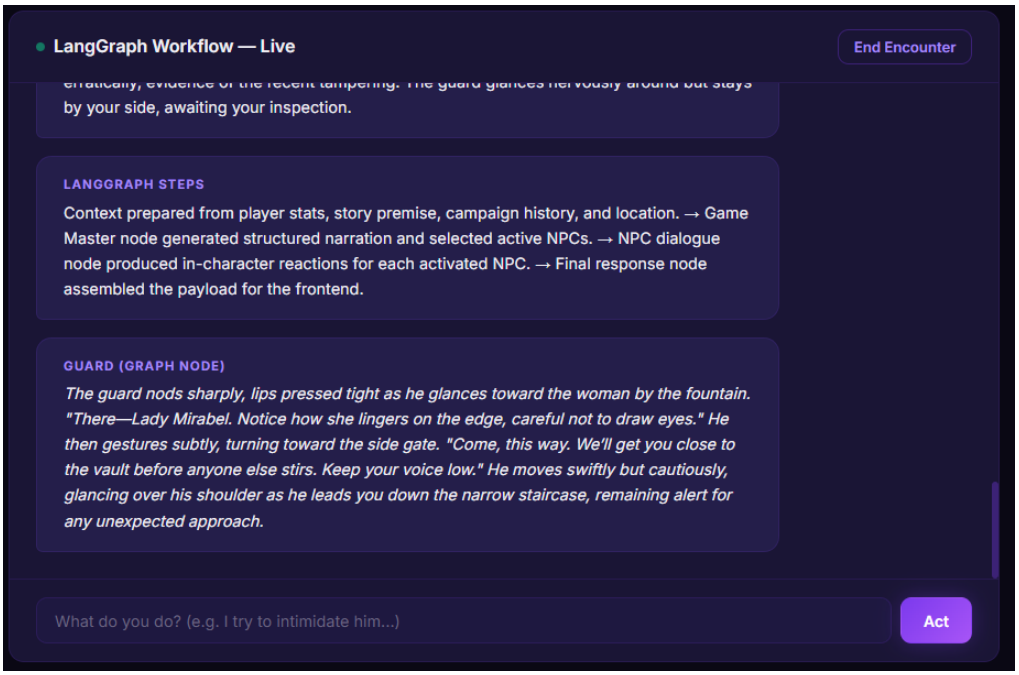

Figure 6. Third interaction where the player asks to identify Lady Mirabel and inspect the vault. The narration shows scene progression from the courtyard to the underground vault area.

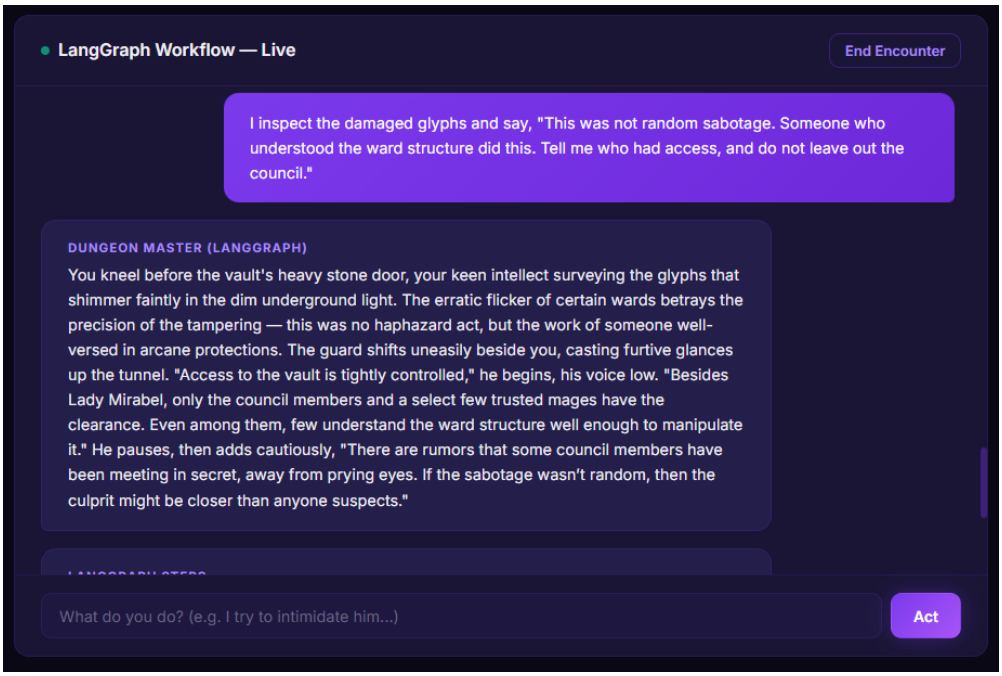

Figure 7. Continuation of the third interaction with LangGraph steps and Guard node output, showing that the same frontend can render narration, workflow, and NPC reaction together.

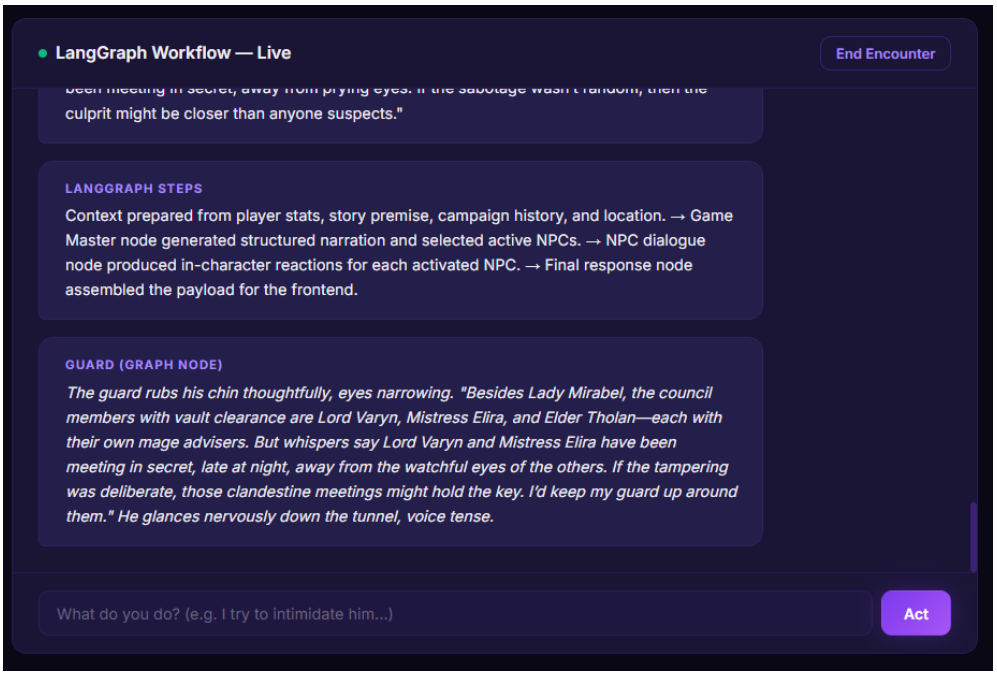

Figure 8. Continuation of the vault investigation showing LangGraph steps and the Guard node's detailed response about additional suspects. This supports the claim that the graph preserves context across multiple turns and exposes the execution path clearly.

9. Implemented Features Summary

- LangGraph was added to the existing MuseAI project.
- CrewAI and LangGraph both remain operational in the same system.
- The encounter state was modeled explicitly for graph execution.
- Conditional branching was implemented for NPC activation.
- Workflow steps are returned to the frontend for visibility.
- LangSmith configuration was prepared for tracing and debugging.
- Git commit history documents the LangGraph integration clearly.

10. Class Discussion Notes

- Why LangGraph was a good fit for state-based encounter orchestration.
- How the graph separates context preparation, narration, branching, and final payload generation.
- Why CrewAI was preserved and not removed.
- How the same frontend can compare two different orchestration strategies.
- How LangSmith can help debug graph execution and branching behavior.

11. Conclusion

The assignment requirements were satisfied by integrating LangGraph into an already existing project, keeping CrewAI active, defining the workflow steps clearly, documenting the work in Git, and preparing screenshot evidence for records. The final result is a hybrid encounter system where the same role-playing scenario can be orchestrated by either CrewAI or LangGraph inside one consistent application.

Prepared for submission as a PDF record with screenshots and implementation notes.